Detecting Machine-Generated Text, Project Code

1. Data preprocessing code

```
import os

os.environ.setdefault('HF_HOME', '/media/filwel/MLProject1/hf_cache')
os.environ.setdefault('HF_HUB_DISABLE_SYMLINKS', '1')
os.environ.setdefault('PYTORCH_CUDA_ALLOC_CONF', 'expandable_segments:True')
os.environ.setdefault('TOKENIZERS_PARALLELISM', 'false')

import gc
import hashlib
import json
import re
import shutil
import time
import warnings
from pathlib import Path

import numpy as np
import pandas as pd
import torch

warnings.filterwarnings('ignore')
torch.backends.cuda.matmul.allow_tf32 = True
torch.backends.cudnn.allow_tf32 = True

PROJECT_DIR = Path('/media/filwel/All/Sakib/Semester 10/ NATURAL LANGUAGE PROCESSING /Project ')
FINAL_DIR = Path('/media/filwel/All/Sakib/Semester 10/ NATURAL LANGUAGE PROCESSING /Final')

PS_DIR = FINAL_DIR / 'experiments' / 'paper_scale'
WORK_DIR = PS_DIR / 'work'
RESULTS_DIR = PS_DIR / 'results'
PROBS_DIR = PS_DIR / 'probs'
MODELS_DIR = PS_DIR / 'models'
CKPT_DIR = Path('/media/filwel/MLProject1/nlp_paper_ckpt')
for d in (WORK_DIR, RESULTS_DIR, PROBS_DIR, MODELS_DIR, CKPT_DIR):
    d.mkdir(parents=True, exist_ok=True)

MAX_LEN = 128
EPOCHS = 5
WARMUP_RATIO = 0.1
PATIENCE = 2
SPLIT_SEED = 42
TRAIN_SEED = 42

MODELS = {'BERT': 'bert-base-uncased', 'DeBERTa': 'microsoft/deberta-v3-base'}
DATASET_NAMES = {'D1': 'DAIGT V2', 'D2': 'HC3'}

print('torch', torch.__version__, '| cuda', torch.cuda.is_available())

from sklearn.model_selection import GroupShuffleSplit


def normalise(t):
    return re.sub(r'\s+', ' ', str(t)).strip()


def content_hash(series):
    return series.map(lambda t: hashlib.md5(normalise(t).lower().encode()).hexdigest())


def balance(df, seed=SPLIT_SEED):
    n = int(df['label'].value_counts().min())
    parts = [df[df['label'] == v].sample(n=n, random_state=seed)
             for v in sorted(df['label'].unique())]
    return pd.concat(parts).sample(frac=1, random_state=seed).reset_index(drop=True)


def load_D1():
    raw = pd.read_csv(PROJECT_DIR / 'daigt.csv')
    df = raw[['text', 'label']].dropna()
    df['label'] = df['label'].astype(int)
    del raw
    gc.collect()
    return balance(df)


def load_D2():
    raw = pd.read_json(PROJECT_DIR / 'hc3.jsonl', lines=True)
    human = raw[['human_answers']].explode('human_answers').rename(
        columns={'human_answers': 'text'})
    human['label'] = 0
    bot = raw[['chatgpt_answers']].explode('chatgpt_answers').rename(
        columns={'chatgpt_answers': 'text'})
```

```
    bot['label'] = 1
    df = pd.concat([human, bot], ignore_index=True).dropna()
    df['text'] = df['text'].astype(str)
    del raw, human, bot
    gc.collect()
    return balance(df)


LOADERS = {'D1': load_D1, 'D2': load_D2}


def group_split(df, seed=SPLIT_SEED):
    groups = df['hash'].values
    gss1 = GroupShuffleSplit(n_splits=1, test_size=0.2, random_state=seed)
    tr_full, te = next(gss1.split(df, df['label'], groups))
    sub = df.iloc[tr_full]
    gss2 = GroupShuffleSplit(n_splits=1, test_size=0.1, random_state=seed)
    tr_rel, val_rel = next(gss2.split(sub, sub['label'], sub['hash'].values))
    idx_tr, idx_val = sub.index.values[tr_rel], sub.index.values[val_rel]
    idx_te = df.index.values[te]
    g_tr = set(df.loc[idx_tr, 'hash'])
    g_val = set(df.loc[idx_val, 'hash'])
    g_te = set(df.loc[idx_te, 'hash'])
    assert not (g_tr & g_val) and not (g_tr & g_te) and not (g_val & g_te)
    return idx_tr, idx_val, idx_te


def build_or_load_splits(tag, rebuild=False):
    data_p = WORK_DIR / f'data_{tag}.parquet'
    split_p = WORK_DIR / f'split_{tag}.npz'
    if not rebuild and data_p.exists() and split_p.exists():
        df = pd.read_parquet(data_p)
        sp = np.load(split_p)
        return df, {'train': sp['train'], 'val': sp['val'], 'test': sp['test']}
    df = LOADERS[tag]()
    df['hash'] = content_hash(df['text'])
    idx_tr, idx_val, idx_te = group_split(df)
    df[['text', 'label']].to_parquet(data_p, index=True)
    np.savez(split_p, train=idx_tr, val=idx_val, test=idx_te)
    return df[['text', 'label']], {'train': idx_tr, 'val': idx_val, 'test': idx_te}


DATA, SPLITS = {}, {}
for tag in ('D1', 'D2'):
    DATA[tag], SPLITS[tag] = build_or_load_splits(tag)
    n = {k: len(v) for k, v in SPLITS[tag].items()}
    total = sum(n.values())
    print(f'{tag} {DATASET_NAMES[tag]:9s} total={total:6d}  train={n["train"]}  '
          f'val={n["val"]}  test={n["test"]}')

import nltk
from nltk.corpus import stopwords
from nltk.stem import WordNetLemmatizer
from nltk.tokenize import word_tokenize

for pkg in ('punkt', 'punkt_tab', 'stopwords', 'wordnet', 'omw-1.4'):
    nltk.download(pkg, quiet=True)

lemmatizer = WordNetLemmatizer()
stop_words = set(stopwords.words('english'))


def preprocess_classical(text):
    text = re.sub(r'[^a-z\s]', ' ', str(text).lower())
    tokens = word_tokenize(text)
    tokens = [lemmatizer.lemmatize(t) for t in tokens
              if t not in stop_words and len(t) > 1]
    return ' '.join(tokens)


demo = DATA['D2']['text'].iloc[0]
print('raw       ', repr(demo[:120]))
print('normalised', repr(normalise(demo)[:120]))
print('classical ', repr(preprocess_classical(demo)[:120]))

from datasets import Dataset
from transformers import AutoTokenizer

_TOKCACHE, _DATACACHE = {}, {}


def get_tokenizer(model_key):
    if model_key not in _TOKCACHE:
        _TOKCACHE[model_key] = AutoTokenizer.from_pretrained(MODELS[model_key])
    return _TOKCACHE[model_key]


def get_tokenized(tag, model_key):
    key = (tag, model_key)
    if key in _DATACACHE:
        return _DATACACHE[key]
```

```
    df, splits = DATA[tag], SPLITS[tag]
    tok = get_tokenizer(model_key)
    parts = {}
    for split, idx in splits.items():
        sub = df.loc[idx]
        ds = Dataset.from_dict({'text': [normalise(t) for t in sub['text']],
                                'labels': [int(v) for v in sub['label']]})
        parts[split] = ds.map(
            lambda b: tok(b['text'], truncation=True, max_length=MAX_LEN),
            batched=True, remove_columns=['text'])
    _DATACACHE[key] = (parts, splits)
    gc.collect()
    return _DATACACHE[key]
```

2. BERT code where the best performance was achieved

```
from sklearn.metrics import (accuracy_score, confusion_matrix,
                             precision_recall_fscore_support)
from transformers import (AutoModelForSequenceClassification,
                          DataCollatorWithPadding, EarlyStoppingCallback,
                          Trainer, TrainingArguments, set_seed)


def weighted_metrics(y, p):
    acc = accuracy_score(y, p)
    pre, rec, f1, _ = precision_recall_fscore_support(
        y, p, average='weighted', zero_division=0)
    return acc, pre, rec, f1


def compute_metrics(eval_pred):
    logits, labels = eval_pred
    preds = np.argmax(logits, axis=-1)
    acc, pre, rec, f1 = weighted_metrics(labels, preds)
    return {'accuracy': acc, 'precision': pre, 'recall': rec, 'f1': f1}


def run_key(tag, model_key, cfg, seed=TRAIN_SEED):
    return (f'full_{tag}_{model_key}_lr{cfg["lr"]:g}_bs{cfg["bs"]}'
            f'_wd{cfg["wd"]:g}_s{seed}')


def train_one(tag, model_key, cfg, seed=TRAIN_SEED, save_model=True):
    learning_rate = cfg['lr']
    batch_size = cfg['bs']
    weight_decay = cfg['wd']
    key = run_key(tag, model_key, cfg, seed)
    run_dir = CKPT_DIR / key
    parts, splits = get_tokenized(tag, model_key)
    tok = get_tokenizer(model_key)

    set_seed(seed)
    model = AutoModelForSequenceClassification.from_pretrained(
        MODELS[model_key], num_labels=2)
    model.config.id2label = {0: 'human', 1: 'ai'}
    model.config.label2id = {'human': 0, 'ai': 1}

    args = TrainingArguments(
        output_dir=str(run_dir),
        learning_rate=learning_rate,
        per_device_train_batch_size=batch_size,
        per_device_eval_batch_size=64,
        weight_decay=weight_decay,
        num_train_epochs=EPOCHS,
        warmup_ratio=WARMUP_RATIO,
        lr_scheduler_type='linear',
        optim='adamw_torch',
        bf16=torch.cuda.is_bf16_supported(),
        eval_strategy='epoch',
        save_strategy='epoch',
        save_total_limit=1,
        load_best_model_at_end=True,
        metric_for_best_model='eval_f1',
        greater_is_better=True,
        logging_steps=200,
        seed=seed,
        data_seed=seed,
        dataloader_num_workers=0,
        report_to='none')

    trainer = Trainer(
        model=model, args=args, train_dataset=parts['train'],
        eval_dataset=parts['val'], data_collator=DataCollatorWithPadding(tok),
        compute_metrics=compute_metrics,
        callbacks=[EarlyStoppingCallback(early_stopping_patience=PATIENCE)])

    t0 = time.time()
    trainer.train()
```

```
    train_secs = time.time() - t0

    out = {'key': key, 'dataset': tag, 'dataset_name': DATASET_NAMES[tag],
           'model': model_key, 'checkpoint': MODELS[model_key],
           'lr': learning_rate, 'batch_size': batch_size,
           'weight_decay': weight_decay, 'seed': seed, 'max_len': MAX_LEN,
           'n_train': len(splits['train']), 'n_val': len(splits['val']),
           'n_test': len(splits['test']), 'train_seconds': round(train_secs, 1),
           'epochs_run': int(trainer.state.epoch or 0)}

    probs = {}
    for split in ('val', 'test'):
        pred = trainer.predict(parts[split])
        raw = pred.predictions[0] if isinstance(pred.predictions, tuple) else pred.predictions
        p = torch.softmax(torch.tensor(raw, dtype=torch.float32), dim=-1).numpy()
        y = np.asarray(pred.label_ids)
        acc, pre, rec, f1 = weighted_metrics(y, p.argmax(1))
        out[split] = {'accuracy': round(acc, 4), 'precision': round(pre, 4),
                      'recall': round(rec, 4), 'f1': round(f1, 4)}
        out[f'{split}_confusion'] = confusion_matrix(y, p.argmax(1)).tolist()
        probs[f'{split}_probs'] = p
        probs[f'{split}_labels'] = y

    np.savez(PROBS_DIR / f'{key}.npz', **probs)
    json.dump(out, open(RESULTS_DIR / f'{key}.json', 'w'), indent=2)

    if save_model:
        mdir = MODELS_DIR / f'{tag}_{model_key}'
        trainer.save_model(str(mdir))
        tok.save_pretrained(str(mdir))
        json.dump(out, open(mdir / 'run_info.json', 'w'), indent=2)

    del trainer, model
    gc.collect()
    if torch.cuda.is_available():
        torch.cuda.empty_cache()
    shutil.rmtree(run_dir, ignore_errors=True)
    print(f'{key}  val_f1={out["val"]["f1"]:.4f}  test_f1={out["test"]["f1"]:.4f}  '
          f'{train_secs / 60:.1f} min')
    return out

BERT_BEST = {
    'D1': {'lr': 3e-5, 'bs': 32, 'wd': 0.1},
    'D2': {'lr': 2e-5, 'bs': 16, 'wd': 0.1},
}

BERT_RESULT, BERT_CFG = {}, {}
for tag in ('D1', 'D2'):
    cfg = BERT_BEST[tag]
    print(f'{tag} {DATASET_NAMES[tag]:9s} BERT  learning_rate={cfg["lr"]:g}  '
          f'batch_size={cfg["bs"]}  weight_decay={cfg["wd"]:g}')
    BERT_CFG[tag] = dict(cfg, key=run_key(tag, 'BERT', cfg))
    BERT_RESULT[tag] = train_one(tag, 'BERT', cfg)
```

3. BERT variant code, DeBERTa, where the best performance was achieved

```
DEBERTA_BEST = {
    'D1': {'lr': 3e-5, 'bs': 16, 'wd': 0.01},
    'D2': {'lr': 3e-5, 'bs': 16, 'wd': 0.1},
}

DEBERTA_RESULT, DEBERTA_CFG = {}, {}
for tag in ('D1', 'D2'):
    cfg = DEBERTA_BEST[tag]
    print(f'{tag} {DATASET_NAMES[tag]:9s} DeBERTa  learning_rate={cfg["lr"]:g}  '
          f'batch_size={cfg["bs"]}  weight_decay={cfg["wd"]:g}')
    DEBERTA_CFG[tag] = dict(cfg, key=run_key(tag, 'DeBERTa', cfg))
    DEBERTA_RESULT[tag] = train_one(tag, 'DeBERTa', cfg)
```

4. Ensemble code

```
from scipy.stats import binomtest


def load_probs(key):
    z = np.load(PROBS_DIR / f'{key}.npz')
    return {k: z[k] for k in z.files}


def paired_test(y, pred_a, pred_b, n_boot=10000, seed=SPLIT_SEED):
    rng = np.random.default_rng(seed)
    wa, wb = pred_a != y, pred_b != y
    b = int((~wa & wb).sum())
```

```
    c = int((wa & ~wb).sum())
    p = binomtest(b, b + c, 0.5).pvalue if (b + c) else 1.0
    idx = rng.integers(0, len(y), size=(n_boot, len(y)))
    boot = wa[idx].mean(1) - wb[idx].mean(1)
    lo, hi = np.percentile(boot, [2.5, 97.5])
    return {'b': b, 'c': c, 'p': float(p),
            'diff_pp': float((wa.mean() - wb.mean()) * 100),
            'ci_lo_pp': float(lo * 100), 'ci_hi_pp': float(hi * 100)}


WEIGHTS = np.round(np.arange(0.0, 1.0001, 0.05), 2)
ENSEMBLE = {}

for tag in ('D1', 'D2'):
    pb = load_probs(BERT_CFG[tag]['key'])
    pdb = load_probs(DEBERTA_CFG[tag]['key'])
    assert np.array_equal(pb['val_labels'], pdb['val_labels'])
    assert np.array_equal(pb['test_labels'], pdb['test_labels'])
    yv, yt = pb['val_labels'], pb['test_labels']

    val_f1 = []
    for w in WEIGHTS:
        mix = w * pb['val_probs'] + (1 - w) * pdb['val_probs']
        val_f1.append(weighted_metrics(yv, mix.argmax(1))[3])
    best_w = float(WEIGHTS[int(np.argmax(val_f1))])

    ens_pred = (best_w * pb['test_probs'] + (1 - best_w) * pdb['test_probs']).argmax(1)
    acc, pre, rec, f1 = weighted_metrics(yt, ens_pred)

    members = {'BERT': pb['test_probs'].argmax(1),
               'DeBERTa': pdb['test_probs'].argmax(1)}
    mem_f1 = {k: weighted_metrics(yt, v)[3] for k, v in members.items()}
    stronger = max(mem_f1, key=mem_f1.get)
    pt = paired_test(yt, ens_pred, members[stronger])

    ENSEMBLE[tag] = {'weight_bert': best_w, 'weight_deberta': round(1 - best_w, 2),
                     'test_f1': round(f1, 4), 'test_accuracy': round(acc, 4),
                     'member_f1': {k: round(v, 4) for k, v in mem_f1.items()},
                     'stronger_member': stronger, 'paired': pt,
                     'degenerate': best_w in (0.0, 1.0),
                     'confusion': confusion_matrix(yt, ens_pred).tolist()}

    print(f'{tag} {DATASET_NAMES[tag]}')
    print(f'   weight_bert={best_w:.2f}  weight_deberta={1 - best_w:.2f}')
    print(f'   BERT {mem_f1["BERT"]:.4f}   DeBERTa {mem_f1["DeBERTa"]:.4f}   '
          f'ensemble {f1:.4f}')
    print(f'   McNemar p={pt["p"]:.4g}  error diff {pt["diff_pp"]:+.3f} pp  '
          f'95 pct CI [{pt["ci_lo_pp"]:+.3f}, {pt["ci_hi_pp"]:+.3f}]')
```